Computer science (code 30424) a.y. 2024-2025

Practice session 5

The following exercises refer to the topics covered in the previous lessons. Some of them will be carried out together with the teacher. Others will have to be carried out individually, and - at the end of the allotted time - the teacher will illustrate and comment on their correct execution. Exercises that will not be carried out in the classroom must in any case be considered an integral part of the Practice, to be carried out independently at home.

The Tutors present in the classroom during the Practice can only provide support in the execution of some steps. Any doubts about the approach used by the teacher and the validity of any alternative solutions should be clarified exclusively with the teacher.

Exercise 1

The file **Exercise 1.py** contains a list called *econ_terms* with some of the most common economic and financial terms. You now have to create the function **wiki1** that must:

- have a sequence of words as a mandatory parameter (it is possible to use the list *econ_terms*, but it must be possible to pass to the function any other sequence)
- choose randomly one word from the list passed as an argument
- show on screen the text "The chosen word is XXXX", where "XXXX" is the city randomly picked from the sequence, asking the user if he/she wants to see the related Wikipedia page in the default web browser. If the answer is affirmative, the program must open the web page, otherwise it must show a thank you message on screen

Then create a new function called **wiki2** that – based on **wiki1** function – creates a dictionary with some text strings as keys and Wikipedia addresses as values. In particular, the function must:

- have a sequence of words and an integer as mandatory parameters (it is possible to use the list *econ_terms*, but it must be possible to pass to the function any other sequence)
- create a new dictionary named *chosen_words* with a number of key-value pairs equal to the integer passed to the function as a mandatory argument, in which keys are words chosen randomly from the sequence passed as a mandatory argument and the values are their Wikipedia pages. **Note**: the dictionary must not contain duplicate keys
- returns the list *chosen_words* dictionary

Exercise 2

The file **Exercise 2.py** contains a Python function that creates a new password at least 8 characters long with at least a letter (uppercase or lowercase), a number and a special character. The function is incomplete and must:

- receive an integer representing the new password length as a mandatory parameter
- set the password length to 8 if the integer passed to the function as a mandatory parameter is less than 8
- to make sure to have a password with at least one letter, one number and one special character, select the first three random elements from each of the three variables (letters, numbers and special_char)
- complete the new password selecting the remaining random elements from the letters, numbers and special_char variables
- return the new password

Exercise 3

The file **Exercise 3.py** contains a Python function that shows on screen a numbered list with all the definitions of the "MyWord" argument, using the WordNet language dataset in NLTK.

Following the same approach, you are asked to create the function *alternatives* that shows on screen as a numbered list all the synonyms of the argument "MyWord" (see the result with the word "happy" below):

```
>>> alternatives('happy')
1. happy
2. felicitous
3. happy
4. glad
5. happy
6. happy
7. well-chosen
```

Exercise 4

In the file **Exercise 4.py** you can find the SentimentIntensityAnalyzer object *MySent* containing the sentiment lexicon and various methods to analyze the text, and the *sentences* list with various sentences on Python. You are asked to create the *analysis* function that, given a sequence of text strings as a mandatory parameter, shows on screen (on different lines) each sentence and its sentiment scores (as a printed dictionary object).

Exercise 5

The file you need to use is called **Exercise 5.py** and contains a tuple with the first semester subjects of the first year of an undergraduate school. You are asked to complete the script creating a program aimed to store in a new Excel file the grades earned in those subjects.

The program – using the objects available in the openpyxl module – must:

- ask the user to enter (without the extension) the name to assign to the new Excel file to create and the name to assign to the single worksheet that will be generated by default

- create a new Excel workbook object assigning to the active worksheet the name entered by the user and inserting the labels "Exam" and "Score" respectively in cells A1 and B1
- ask the user to enter – one by one (by column) – the scores of each exam, starting from row 2. In particular, column A will have to show the names of the courses and column B will have to host the correspondent scores entered by the user
- in the second row below the name of the last course (column A) must then be inserted the label “Average” (leaving therefore a blank row between the last course and the label “Average”)
- in the corresponding cell on the right (column B) must be shown the average value of the scores, calculated using the appropriate function available in Excel (as a string, respecting the syntax and the language of your version of Excel)
- save the Excel workbook using the name initially entered by the user and adding the extension “.xlsx”
- Print on the screen the confirmation message: “The creation of the file FileName.xlsx has been completed. Thank you!” (on two lines)